48. Neural Networks

Neural Networks: Connecting the Dots in Data

Previous Section:

47. T-Distributed Stochastic Neighbor Embedding

Contents:

Neural Networks. You’ve probably heard this term everywhere. Machine Learning, Deep Learning, Computer Vision, Natural Language Processing. It shows up again and again. And usually, it’s associated with the most powerful systems we have today: Image recognition; Language translation; Voice assistants; Autonomous driving.

So naturally, it sounds important. But here’s the problem. Most learners encounter the term *neural network* long before they actually understand what it means. They hear phrases like: “Inspired by the human brain”; “Layers of neurons”; “Deep learning models”

And somehow, that's supposed to explain everything. But it doesn't. Because saying "it works like the brain" is not an explanation. It's just a metaphor. If we rely only on that metaphor, neural networks start to feel mysterious. Almost like a black box: Data goes in → Predictions come out → And something complicated happens in between.

But you're not here for mystery. You're here to understand. So let's approach this the same way we approached Machine Learning. Not by jumping straight into formulas. But by asking a simpler question:

> ***What problem are neural networks actually trying to solve?***

Because once you understand that, everything else starts to make sense. And the answer begins with something fundamental: The type of data we are dealing with. Not all data is clean, structured, and easy to work with. In fact, most real-world data is not. And that's exactly where neural networks come in.

1. Neural Networks: Working with Unstructured Data

Before we understand neural networks, we need to understand something more fundamental: **The type of data we are dealing with.** Because not all data behaves the same. And this difference? It's exactly why neural networks exist.

Structured Data — "Everything is in its place"

Let's start with the easier case. Structured data is clean, organized, predictable, well-defined. It usually looks like a table:

- Rows → individual records
- Columns → features (variables)

Each value has a clear meaning. Each column has a fixed role. Think about: Spreadsheets; Databases; CSV files; Tables of numbers. Everything is neatly arranged. Because of this structure... machines love it. Why? Because: Relationships are explicit → Formats are consistent → Data is easy to query and analyze

This is the kind of data most traditional Machine Learning models were designed for. Linear Regression, Decision Trees, Support Vector Machines... They work extremely well here. Because the problem is clear. The structure is clear. And the patterns? Often easier to detect.

Unstructured Data — "The real world is messy"

Now let's look at reality. Most data in the real world doesn't look like a table. It looks like this: An image; A voice recording; A paragraph of text; A video; A social media post. This is **unstructured data**. And unlike structured data, there are no neat rows. No predefined columns. No obvious features. Just raw information.

A single image, for example, is just a grid of pixels. But where is the "face"? Where is the "object"? Where is the "meaning"? It's not explicitly written anywhere. The structure exists, but it is hidden. And that's the problem.

The Data Paradox: Here's something interesting. In the real world: Most data is **unstructured.** But most traditional models work best on **structured data.** So we have a mismatch. The data we *have,* is not the data our models were designed for. Structured data is easier to analyze, but less common. Unstructured data is everywhere, but much harder to use. And this creates a challenge.

Why Traditional Models Struggle?

Throughout Machine Learning, we've seen many powerful models: Linear models; Tree-based models; Support Vector Machines; Clustering algorithms; Probabilistic models. These are not weak models. They are extremely effective. But they rely on something important: **Well-defined features.**

In structured data, features are given to us. In unstructured data? We have to find them. And that's where things break. Because extracting features from: Images; Text; Audio is not simple. It requires domain knowledge, manual effort, a lot of trial and error. And even then, we might miss important patterns.

Enter Neural Networks

This is where neural networks change the game. Instead of asking: "What features should we use?" → They ask: ***"Can the model learn the features by itself?"***

The answer is: Yes. Neural networks are designed to learn directly from raw data. No need to manually define everything. They take input like: Pixels; Words; Sound waves. And through multiple layers, they begin to discover

patterns. Simple patterns first: Edges in images; Basic sounds; Word fragments. Then more complex ones: Shapes; Objects; Meaning

This is called **hierarchical learning**. Layer by layer, the model builds understanding.

This ability changes everything. Because now: We are no longer limited to clean, structured data. We can work directly with real-world information. Images. Speech. Language. Video. Things that were once difficult become possible.

So neural networks are not just "another model." → They are a response to a fundamental problem: ***How do we learn from messy, unstructured data?***

And in the next section, we'll start opening that black box. Not as a mystery. But as a system you can actually understand.

2. Neural Networks: The History and Core Concepts

Let's start with the name itself. **Neural Network.** It sounds biological, because it is. The idea comes from the human brain.

The Inspiration — "Learning from the Brain"

Your brain is made of billions of neurons. Each neuron receives signals, processes them, then sends signals forward. Individually, a neuron is simple. But together? They form an incredibly powerful system. A system capable of: Recognizing faces; Understanding language; Learning from experience

So naturally, researchers asked a bold question: ***"What if we could build something similar... artificially?"***

The Early Ideas — Simple but Powerful

Back in 1943, two researchers, Warren McCulloch and Walter Pitts, proposed a simple model of a neuron. Not biological, but mathematical. A unit that takes inputs, applies a rule, produces an output. It was simple. Very simple. But it introduced a powerful idea: Intelligence could be modeled using networks of simple units.

A few years later, in 1949, Donald Hebb added something even more important. A learning principle. His idea was: *"Neurons that fire together... wire together"*. In other words: Connections strengthen through use. This became one of the foundations of learning in neural networks.

The First Attempts — And Early Struggles

As computers emerged in the 1950s. Researchers tried to turn these ideas into reality. Some early attempts failed. Computers were simply not powerful enough. But progress continued.

In 1959, two researchers, Bernard Widrow and Marcian Hoff, developed systems called: ADALINE and MADALINE. These were among the first practical neural networks. And for the first time... Neural networks were applied to real-world problems: Filtering noise in phone lines, predicting signals → Simple tasks, but a huge step forward.

They also introduced learning rules. Ways to adjust the network when it makes mistakes. Because learning is not magic. It is adjustment. Small corrections repeated many times.

The Decline — When Things Went Wrong

Then something interesting happened. Research slowed down. Not because the idea was wrong. But because of limitations. At the time: Hardware was weak; Models were too simple; Expectations were too high

There was also a critical issue: Early neural networks struggled with **multi-layer learning**. And without multiple layers. Their power was very limited.

At the same time, another approach dominated computing: Traditional architectures (von Neumann systems). They were faster. More reliable. More practical. So neural networks were... mostly abandoned.

The Comeback — Quiet Progress

Even during this period, some ideas continued to develop. In the 1970s: New network structures appeared. Multi-layer networks were explored. Slowly, the field evolved. Not with hype. But with persistence.

From Biology to Mathematics

Now here's the important shift. Artificial Neural Networks are **not real brains**. They are simplified mathematical models. Instead of biological neurons, we use **artificial neurons (nodes)**. Each node receives numerical inputs, applies a mathematical function, produces an output. And when we connect many of them together, we get a network.

How a Neural Network Learns?

A neural network learns by adjusting connections. Not physically, but mathematically. Each connection has a value: A weight. During training: The network makes predictions, compares them with the correct answer, adjusts the weights. Step by step, it improves. Not by memorizing, but by shaping itself to match the data.

What Is a Neural Network?

Now we can finally answer it properly. A neural network is: **A system of interconnected computational units designed to learn patterns from data and inspired by how the brain processes information.** It is one of the core models in Machine Learning, Deep Learning, and more. And its strength comes from one key ability: Learning complex relationships.

Linear vs Non-Linear Thinking

To understand why neural networks matter, we need to look at a limitation of earlier models. Some problems are **linear**. Which means: You can separate data using a straight line. These are easier. Many classical models work well here.

But many real-world problems are not like that. They are **non-linear**. Which means the boundary is curved, complex. Sometimes impossible to describe with a simple rule.

Think about: Recognizing faces; Understanding speech; Classifying images. You cannot solve these with a straight line. And this is where neural networks shine. By stacking layers, they can learn simple patterns first, then combine them into complex ones. This allows them to model: Curves; Shapes; Deep relationships

Sometimes data looks messy. Classes overlap. Everything seems mixed. But that doesn't always mean the problem is complex. It might just mean: The features are not good enough. So before jumping to neural networks, we must ask: "*Is the problem truly non-linear? → Or are we missing the right representation?*"

Why Neural Networks Changed Everything?

The biggest advantage of neural networks is this: They learn features automatically. Instead of manually designing inputs. They discover patterns on

their own through layers, through data, through training.

So in the end, neural networks are not just about "mimicking the brain". They are about solving a deeper problem: ***"How do we learn complex patterns... directly from raw data?"***

And in the next section, We'll open the system itself. Not just the idea. But the structure.

3. Neural Networks: Key Components

Now that we understand *why* neural networks exist. Let's open the box. Not the full complexity yet. Just the core pieces. Because at its heart, a neural network is not magic. It's a system built from a few simple components.

The Building Blocks - Let's start with the smallest unit.

- **Neurons - "The decision units":** A neural network is made of **neurons** (also called nodes). Each neuron does something very simple: It receives inputs, processes them, then produces an output. That's it. No intelligence on its own. But when many neurons work together, something interesting happens.
- **Connections - "How information flows":** Neurons don't exist in isolation. They are connected. These connections carry information, link one neuron to another, and form the structure of the network Think of it like a network of roads. Signals travel from one point to another.
- **Weights and Biases - "What matters more?":** Not all inputs are equal. Some matter more than others. This is controlled by **weights**. Each connection has a weight: Large weight → strong influence; Small weight → weak influence. And then there is **bias**. Bias allows the neuron to shift its behavior. Without it, the neuron is too limited. With it, the model becomes flexible.
- **Propagation — "How signals move":** Once inputs enter the network, they don't just sit there. They flow. From one layer to the next. At each step: Inputs are combined, transformed, passed forward. This flow is called **propagation**. It's how information travels through the network.
- **Learning Rule — "How the network improves":** Now comes the most important part. Learning. A neural network does not stay fixed. It adjusts itself. Using a **learning rule**, the network looks at its prediction, compares it

with the correct answer, and then updates weights and biases. Over time, it gets better. Not instantly. But gradually.

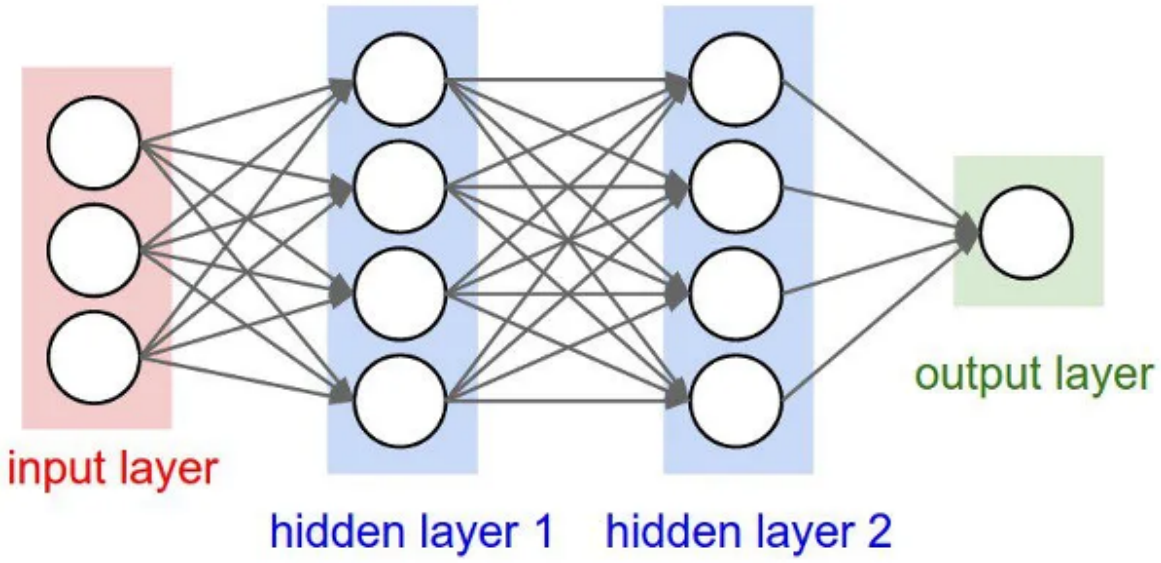
How Learning Actually Happens - Let's simplify the process. Every neural network learns in a loop. Three steps.

- **Step 1 - Input Computation:** We start with data. The input enters the network through the first layer. Each feature becomes a signal.
- **Step 2 - Output Generation:** The network processes the data. Using current weights and biases... it produces a prediction. This could be: A number (regression) or a class in a form of probability (classification)
- **Step 3 - Iterative Refinement:** Now we check how wrong was the prediction. The network calculates the error. And then: Adjusts weights, adjusts biases. This process repeats. Again. And again. Until the model improves.

The Structure of a Neural Network - Now let's zoom out. A full neural network is organized into layers.

- **Input Layer - "Where data enters":** This is the starting point. Each neuron represents one feature. Nothing complex happens here. It simply passes data forward.
- **Hidden Layers - "Where learning happens":** This is the core. The real work happens here. Neurons combine inputs, apply transformations, learn patterns. And most importantly, they use **non-linearity.** This is what allows the network to learn complex relationships.
- **Output Layer - "The final answer":** This is the end. The network produces its result: A predicted value; A probability; A decision
- **Connections — "The arrows between neurons":** If you look at a neural network diagram... you'll see many arrows. These arrows represent **connections**. They carry information from one neuron to another, This is how signals move through the network.
- **Weights — "How strong is the influence?":** Each connection is not equal. Every connection has a **weight**. This weight determines how strongly one neuron influences another. During training these weights are adjusted. So the network can better capture patterns in the data.

(The following image represents the core structure of a Neural Network)



Putting It All Together - So what do we actually have?

A system where: Neurons process information; Connections carry signals; Weights control importance; Biases add flexibility; Layers build complexity; Learning adjusts everything

And through repetition, the network transforms from random guesses into meaningful predictions.

At this point, nothing here is complicated individually. But when all these pieces interact. That's when neural networks become powerful. And in the next step, we'll go deeper into what actually happens inside a single neuron.

4. Neural Networks: Simplified Workflow

Now that we understand the structure. Let's look at what actually happens inside. Because no matter how large a neural network becomes. The core computation is always the same. Simple steps. Repeated again and again.

Step 1 - Weighted Sum (Linear Transformation)

Everything starts with inputs. A neuron receives values: $x_1, x_2, \ldots, x_n$. Each input has a weight: $w_1, w_2, \ldots, w_n$

Some inputs matter more. Some matter less. The neuron combines them into a single value:

$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

This is called the **weighted sum**. Or linear combination, pre-activation value

The bias b plays an important role. It allows the model to shift. Without bias, the model is too restricted. It cannot fit real-world data properly

This step should feel familiar. It's similar to Linear Regression and Logistic Regression. But here, it happens everywhere. Across many neurons. Across many layers.

Step 2 - Activation Function (Non-Linearity)

After computing z, we transform it. Using an **activation function**:

$$a = f(z)$$

This is where things become powerful. Because without this step, the entire network would behave like a single linear model. No matter how many layers we stack.

Activation functions introduce **non-linearity**. Which allows the network to learn curves, complex boundaries, deep relationships.

Some common activation functions:

- **Sigmoid**: Output between 0 and 1. Useful for binary classification

$$\text{Sigmoid:} \quad f(z) = \frac{1}{1 + e^{-z}}$$

- **Softmax:** Output a probability distribution. Useful for multi-class classification

$$\text{Softmax} = \frac{\sum e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

- **Tanh:** Output between -1 and 1. Often converges faster than sigmoid

$$f(z) = \tanh(z)$$

- **ReLU (Rectified Linear Unit):** Outputs 0 if negative, Outputs z if positive. Simple. Fast → Widely used in modern networks.

$$ReLU = max(0, z)$$

Step 3 - Propagation Through Layers

Once we compute the activation, it doesn't stop there. It becomes input for the next layer. This process is called: **Forward propagation.** Mathematically:

$$a^{(l)} = f\left(W^{(l)}a^{(l-1)} + b^{(l)}\right)$$

Where:

- $a^{(l)}$: output of current layer
- $W^{(l)}$: weights
- $b^{(l)}$: bias
- $a^{(l-1)}$: input from previous layer

And this repeats. Layer by layer. Until we reach the final output.

Hierarchical Learning — "From simple to complex"

Here's where it gets interesting. Each layer learns something different.

For example, in image recognition:

- Early layers → edges, textures
- Middle layers → shapes, patterns
- Deep layers → objects, meaning

This is called: **Hierarchical learning.** And it's one of the biggest strengths of neural networks.

Step 4 — Output Layer

Finally... we reach the output. But the form of the output depends on the problem.

- **Regression — "Predict a number"**
  - Output: a single value → Example: house price, temperature
  - Usually: No activation or linear activation
- **Classification — "Predict a category"**
  - Output: probabilities
  - Common choices: **Sigmoid (**Binary classification, output between 0 and 1); **Softmax (**Multi-class classification, outputs probability distribution, the highest probability becomes the prediction).

So every neural network, no matter how complex it is, follows the same core workflow:

$$\text{Linear transformation} \rightarrow \text{Activation} \rightarrow \text{Propagation} \rightarrow \text{Output}$$

And during training: ***Repeat*** → ***Adjust*** → ***Improve. Again and again.***

That's the entire system. Simple steps. Repeated at scale. And that's what makes neural networks powerful.

5. Neural Network Works: A Simple Example

Let's make this real. Not with theory. But with actual data. Suppose we have the following dataset - The Iris Dataset:

Sepal Length	Sepal Width	Petal Length	Petal Width	Target
5.1	3.5	1.4	0.2	0
4.9	3.0	1.4	0.2	0
4.7	3.2	1.3	0.2	0
4.6	3.1	1.5	0.2	0
5.0	3.6	1.4	0.2	0
7.0	3.2	4.7	1.4	1
6.4	3.2	4.5	1.5	1
6.9	3.1	4.9	1.5	1
6.3	3.3	6.0	2.5	2

Sepal Length	Sepal Width	Petal Length	Petal Width	Target
5.8	2.7	5.1	1.9	2

Dataset Overview

We are working with a classification problem:

- **Target 0** → Iris setosa (5 samples)
- **Target 1** → Iris versicolor (3 samples)
- **Target 2** → Iris virginica (2 samples)

Each row is a flower. Each column (except target) is a feature.

In this section, we are not solving the neural network. We are not training anything. We are only answering one question: **What values does each layer receive?** Because before understanding learning, we must understand **data flow**.

Step 1 — Input Layer

The input layer represents the features. We have 4 features: Sepal Length; Sepal Width; Petal Length; Petal Width. So we will have: ***4 input neurons***

How Data Enters the Network: Each neuron does not store a single number. It receives values from all samples. So instead of thinking: "one neuron = one value". Think: ***"one neuron = one feature across all samples"***

Feature Representation: Let's rewrite the inputs as vectors.

- **Neuron 1 — Sepal Length**

$$x_1 = [5.1,\ 4.9,\ 4.7,\ 4.6,\ 5.0,\ 7.0,\ 6.4,\ 6.9,\ 6.3,\ 5.8]$$

- **Neuron 2 — Sepal Width**

$$x_2 = [3.5,\ 3.0,\ 3.2,\ 3.1,\ 3.6,\ 3.2,\ 3.2,\ 3.1,\ 3.3,\ 2.7]$$

- **Neuron 3 — Petal Length**

$$x_3 = [1.4,\ 1.4,\ 1.3,\ 1.5,\ 1.4,\ 4.7,\ 4.5,\ 4.9,\ 6.0,\ 5.1]$$

- **Neuron 4 — Petal Width**

$$x_4 = [0.2,\ 0.2,\ 0.2,\ 0.2,\ 0.2,\ 1.4,\ 1.5,\ 1.5,\ 2.5,\ 1.9]$$

A More Compact View - We can also represent everything as a matrix:

$$X = \begin{bmatrix} 5.1 & 3.5 & 1.4 & 0.2 \\ 4.9 & 3.0 & 1.4 & 0.2 \\ 4.7 & 3.2 & 1.3 & 0.2 \\ 4.6 & 3.1 & 1.5 & 0.2 \\ 5.0 & 3.6 & 1.4 & 0.2 \\ 7.0 & 3.2 & 4.7 & 1.4 \\ 6.4 & 3.2 & 4.5 & 1.5 \\ 6.9 & 3.1 & 4.9 & 1.5 \\ 6.3 & 3.3 & 6.0 & 2.5 \\ 5.8 & 2.7 & 5.1 & 1.9 \end{bmatrix}$$

Each row = one sample; Each column = one feature

Target Representation - The target vector:

$$y = [0,\ 0,\ 0,\ 0,\ 0,\ 1,\ 1,\ 1,\ 2,\ 2]$$

At the input layer: No computation happens, no learning happens. It simply holds the data and passes it forward

So when we say: “Neuron 1 stores Feature 1” → What we really mean is: It represents the entire vector of that feature. Not just a single value.

In the next step, these values will be transformed, combined, weighted. And passed into the hidden layers. That’s where things start to get interesting.

Step 2 — Hidden Layers

We mow enter the world of **the hidden layers**. Let’s define the architecture:

- **Hidden Layer 1 → 32 neurons**
- **Hidden Layer 2 → 16 neurons**

Now a natural question appears: **Is it a strict rule that hidden layers must have more neurons than the input layer?**

Answer: No. Not at all.

There is **no rule** that hidden layers must be larger. You can have:

- Fewer neurons (compression)
- More neurons (expansion)
- Same size (neutral transformation)

What matters is not size, but: **What representation the network learns**

Sometimes we expand to capture complex patterns. Sometimes we compress to force abstraction. Sometimes we do both.

Neural networks are not rigid structures, they are **design choices shaped by the problem**.

Step 3 — What Actually Flows Into a Hidden Neuron?

Now we reach the key idea. Each neuron in a hidden layer does **not receive a single feature alone**. Instead, it receives a **weighted combination of ALL input features.**

So instead of thinking:

- Neuron 1 = Sepal Length
- Neuron 2 = Sepal Width

Think correctly: Each neuron sees the **entire input vector**, but reacts differently to it.

Mathematically, each hidden neuron takes all input features, multiplies them by learned weights, adds a bias, then applies an activation function.

So conceptually: A neuron is learning *what combination of features matters*

For example:

- One neuron may focus on petal size patterns
- Another may focus on sepal ratios
- Another may detect nonlinear interactions

Even though throughout its learning process, we don't explicitly tell it anything.

Step 4 — First Hidden Layer (32 Neurons)

When the input matrix enters the first hidden layer:

- Each of the 32 neurons receives the full feature vector
- Each neuron has its own **weight vector**
- Each produces a single transformed value

So instead of: **4 features → 4 numbers**

We now have: **4 features → 32 new learned features**

This is called **Feature transformation (representation learning).** The network is no longer seeing raw flowers. It is seeing learned patterns of flowers.

Step 5 — Second Hidden Layer (16 Neurons)

Now the output of the first hidden layer becomes input to the second:

- 32-dimensional representation → 16 neurons

What happens here? Even more abstraction. If the first layer learned simple patterns (feature interactions). Then the second layer learns combinations of those patterns. So the network is gradually building: **raw data → patterns → higher-level concepts**

Step 6 — Output Layer (3 Neurons)

Now we reach the final layer: **3 neurons.** Each neuron represents a class: ***[Iris setosa, Iris versicolor, Iris virginica]***

But here is the important part: These are not final decisions yet. They are called **logits** (raw scores).

Step 7 — Softmax Activation (Turning Scores into Probabilities)

To convert logits into probabilities, we use: **Softmax function (because we are dealing with a multi-class classification problem).**

It transforms the 3 outputs into: Probability of class 0; Probability of class 1; Probability of class 2

And ensures: All values are between 0 and 1 → All values sum to 1. So the output becomes: ***"How confident is the model in each class?"***

Step 8 — Loss Function (Measuring Mistake)

Now the model compares: Predicted probabilities vs True labels. This produces a value called: **Loss**

The loss answers: “How wrong are we?”

- Higher loss → worse prediction
- Lower loss → better prediction

Step 9 — Backpropagation (Learning from Mistakes)

Now the system does something powerful. It computes: **Gradients.**

These gradients tell: Which weights contributed to the error. How to adjust them. In which direction to improve

Then: weights are updated, biases are updated. And this process flows back: Output layer → hidden layers → input layer

This is called: ***Backpropagation***

Step 10 — Batch Learning

Now here is how data actually moves. We don’t usually feed all samples one by one blindly. We use **batches**. Each neuron processes: one sample, or multiple samples, or a subset of dataset. This subset is called: **Batch**

Now let’s use your dataset:

$$X = \begin{bmatrix} 5.1 & 3.5 & 1.4 & 0.2 \\ 4.9 & 3.0 & 1.4 & 0.2 \\ 4.7 & 3.2 & 1.3 & 0.2 \\ 4.6 & 3.1 & 1.5 & 0.2 \\ 5.0 & 3.6 & 1.4 & 0.2 \\ 7.0 & 3.2 & 4.7 & 1.4 \\ 6.4 & 3.2 & 4.5 & 1.5 \\ 6.9 & 3.1 & 4.9 & 1.5 \\ 6.3 & 3.3 & 6.0 & 2.5 \\ 5.8 & 2.7 & 5.1 & 1.9 \end{bmatrix}$$

So if **batch size = 3**, we do not process all 10 samples at once. We split them like this:

Batch 1 (samples 1–3)

- Sample 1 → $[5.1, 3.5, 1.4, 0.2]$
- Sample 2 → $[4.9, 3.0, 1.4, 0.2]$
- Sample 3 → $[4.7, 3.2, 1.3, 0.2]$

So Batch 1 becomes:

$$X^{(1)} = \begin{bmatrix} 5.1 & 3.5 & 1.4 & 0.2 \\ 4.9 & 3.0 & 1.4 & 0.2 \\ 4.7 & 3.2 & 1.3 & 0.2 \end{bmatrix}$$

Batch 2 (samples 4–6)

$$X^{(2)} = \begin{bmatrix} 4.6 & 3.1 & 1.5 & 0.2 \\ 5.0 & 3.6 & 1.4 & 0.2 \\ 7.0 & 3.2 & 4.7 & 1.4 \end{bmatrix}$$

Batch 3 (samples 7–9)

$$X^{(3)} = \begin{bmatrix} 6.4 & 3.2 & 4.5 & 1.5 \\ 6.9 & 3.1 & 4.9 & 1.5 \\ 6.3 & 3.3 & 6.0 & 2.5 \end{bmatrix}$$

Batch 4 (sample 10 only)

$$X^{(4)} = \begin{bmatrix} 5.8 & 2.7 & 5.1 & 1.9 \end{bmatrix}$$

Inside every batch, the same learning loop is repeated.

So for **one batch**, the process looks like this:

1. The 4 input neurons receive the batch values → each neuron now processes a **vector of values (batch size = 3 samples)**
2. The network performs **forward propagation**

3. It computes predictions for all 3 samples in the batch
4. The **loss is calculated** (usually averaged over the batch)
5. **Backpropagation** is applied once for the entire batch
6. **Weights and biases are updated**

Then the next batch begins: 3 new samples enter, the same process repeats again. This loop continues until all batches in the dataset are processed.

Batch Learning Strategies (How we choose to process data)

There are three main strategies for feeding data into the model:

- **Batch Gradient Descent:** Uses the **entire dataset at once, o**ne update per full dataset pass. Stable updates → ***But slow for large datasets***
- **Stochastic Gradient Descent (SGD):** Uses **one sample at a time** → One update per sample. Very fast updates → ***Very noisy learning path***
- **Mini-batch Gradient Descent (Most common):** Uses a **small group of samples (e.g., 3, 32, 64, 128).** One update per batch → Standard choice in modern neural networks → And what we are using in the above example (**batch size = 3**)

Mini-batch learning works because: Instead of learning from one example or all examples at once, the model learns from **small, meaningful groups of data.** This gives the best balance between: learning stability, computational efficiency, and generalization ability

Step 10 — Epoch (One Full Cycle of Learning)

Now we connect everything together. An **epoch** is defined as:

> One complete pass through the entire dataset

So what actually happens in one epoch?

1. The dataset is split into batches (e.g., batch size = 3)
2. Batch 1 goes through forward → loss → backprop → update
3. Batch 2 does the same
4. Batch 3 does the same

5. Batch 4 (if remaining samples) is processed
6. Once all batches are processed → **one epoch is complete**

Example If we have: 10 samples total → batch size = 3

Then:

- Batch 1 → samples 1–3
- Batch 2 → samples 4–6
- Batch 3 → samples 7–9
- Batch 4 → sample 10

After all four batches are processed: 1 epoch is completed

Why epochs matter? One epoch is usually not enough for learning. So we repeat the process:

- Epoch 1 → model learns initial patterns
- Epoch 2 → continue with the improved weights → refines further
- Epoch 3 → continue to refine even further
- ... and so on

Each epoch makes the model slightly better at capturing the true patterns in the data

- **Batch** → how data is grouped for learning
- **Batch update** → one step of learning
- **Epoch** → one full cycle through all data

So training a neural network is essentially: Repeating batch updates across many epochs until the model converges. This is how a neural network gradually transforms from random weights to meaningful pattern recognition system.

6. How We Actually Implement a Neural Network

Now we leave the theory behind for a moment. Because at this point, a natural question appears:

"All of this sounds good... but how do we actually build it?"

Well, to answer that question: There is more than one way to build a Neural Network

In practice, we don't manually construct every neuron by hand. Instead, we choose tools that already implement the math for us.

So we have multiple levels of implementation:

- **By hand (NumPy)** → full control, full transparency
- **Scikit-learn** → simple models, fast prototyping
- **TensorFlow / Keras** → full deep learning framework

Each one sits at a different level of abstraction. So the real idea is not *how many tools exist,* but: ***"How much of the learning process do you want to control yourself?"***

1. Building it from scratch (NumPy level)

At the lowest level, everything is explicit. You define: Weights; Biases; Forward propagation; Activation functions; Loss function; Backpropagation.

Nothing is hidden. You literally simulate what the network does step by step. Here, you are not using a neural network, you are *becoming* the neural network. This is slow, but extremely important for understanding.

2. **Scikit-learn (Simple Neural Network abstraction)**

Now we move one step higher. In Scikit-learn, you can define a neural network like:

- Input → hidden layers → output
- Training → done in a few lines

You don't manually compute gradients anymore. The system handles: Forward pass; Backpropagation; Optimization

So instead of building logic, you are now configuring it. You are no longer building neurons... you are arranging them.

3. TensorFlow / Keras (Deep Learning world)

Now we reach the most practical level used in real systems. Here, everything becomes modular.

- You define: Architecture (layers); Activation functions; Optimizers (Adam, SGD, etc.); Loss functions; Training process
- And the framework handles: Gradient computation; GPU acceleration; Backpropagation; Batch processing

So your workflow becomes: ***Define*** → ***Compile*** → ***Fit*** → ***Evaluate***

This is where real-world neural networks live.

No matter which tool you use, the internal process never changes:

- Input flows forward → Predictions are made → Error is calculated → Gradients flow backward → Weights are updated

Only the *level of abstraction* changes. But during training, data is not processed all at once. We use:

- **Batch Size** → how many samples we process before updating weights
- **Epoch** → one full pass through the entire dataset.

But one epoch is never enough. Because learning is not a single action. It is repetition with correction.

Summary

Now let's step back. At this point, you have seen Neural Networks from multiple angles:

- Structure (input → hidden → output)
- Transformation (features → representations → probabilities)
- Learning process (loss → gradients → updates)
- Training dynamics (batches and epochs)
- Implementation (NumPy → Scikit-learn → TensorFlow)

But I want to make one thing clear: **A Neural Network is not a magic box.** It is a system that:

- Receives data
- Transforms it step by step

- Measures its mistakes
- Adjusts itself repeatedly

That's it. No mystery. No intelligence hidden inside. Just structured learning from data.

And why does this matter? Because once you understand this foundation. Everything that comes next stops feeling abstract. So when you move into: Multi-Layer Perceptrons You will not see something new. You will see: the same idea, just expanded. More layers. More transformations. More complexity. But the same core principle: Learn → Adjust → Improve

A Neural Network is not about memorizing data. It is about building a system that slowly learns how to represent the world, one update at a time. And this is only the beginning.

References

https://www.geeksforgeeks.org/deep-learning/neural-networks-a-beginners-guide/

https://www.ibm.com/think/topics/neural-networks

https://developers.google.com/machine-learning/crash-course/neural-networks

https://www.datacamp.com/blog/what-are-neural-networks

https://www.youtube.com/watch?v=CqOfi41LfDw

https://www.youtube.com/watch?v=IN2XmBhILt4

Next Section:

🤖 49. Multi-Layer Perceptrons